MidSem Report Design Project

Building, Processing, and Validating a Reddit Dataset for r/kolkata and r/KolkataCity: An Incremental Collection and ETL Pipeline



Marmik Sapovadia
(2023A7PS0057H)

Under Guidance of
Prof. Dipanjan Chakraborty
(BITS Pilani, Hyderabad Campus)

Course Number: CS F377

Contents

ACKNOWLEDGMENTS **2**

ABSTRACT **3**

1 Introduction to Dataset Processing and Validation **4**
1.1 Raw API Data Is Not a Dataset 4
1.2 Why Validation Is a First-Class Concern 4
1.3 This Project 4

2 Aims and Scope **5**

3 Literature Review and Knowledge Acquisition **6**
3.1 Pushshift and the Archive-First Approach to Reddit Data 6
3.2 Idempotent, Re-Derivable Processing 6
3.3 Deduplication by Stable Identifier 6

4 Processing and Validation Methodology **7**
4.1 Raw Data as the Single Source of Truth 7
4.2 Posts and Cross-Posts 7
4.3 Comments 7
4.4 Deduplication by Reddit ID 7
4.5 Derived Tables: Users and Monthly Activity 7
4.6 Validation Against the Collection-State Ledger 8

5 Dataset and Validation Results **9**
5.1 Processed Table Sizes and Headline Statistics 9
5.2 User Participation Across Subreddits 9
5.3 Discussion 9

6 Challenges and Strategic Optimizations **10**
6.1 Reconciling Inconsistent Raw Schemas 10
6.2 Deduplicating Across Independently Resumed Runs 10
6.3 Processing Very Large Tables Without Loading Them Fully Into Memory 10
6.4 Interpreting a Validation Report That Flags Expected Gaps 10

7 Future Work **11**

8 Conclusion **12**

9 Appendix **13**

10 References **15**

ACKNOWLEDGMENTS

I would like to thank my project guide, Prof. Dipanjan Chakraborty, for his continual feedback on the data-processing and validation design decisions described in this report, and for pushing the project toward a dataset that is verifiably correct rather than merely collected.

I am thankful to the Birla Institute of Technology and Science (BITS), Pilani, Hyderabad Campus, for the academic environment and computational resources that made this work possible, and to my peers in the CS F377 course for discussions on deduplication strategy and schema design that shaped the processing pipeline documented here.

I also acknowledge the maintainers of Arctic Shift, the free archival service that supplied the raw data this pipeline processes, and the developers of the `pandas` and `NumPy` libraries, on which every processing and validation script in this project depends. Finally, I acknowledge Baumgartner et al. (2020) and the Pushshift project, whose archival approach Arctic Shift continues and which this project's own validation methodology draws on.

ABSTRACT

Collecting raw posts and comments from a subreddit is only half of building a usable dataset; the other half is turning that raw, API-shaped data into clean, de-duplicated, analysis-ready tables whose correctness can actually be verified. This project is concerned with that second half: the processing and validation layer of a pipeline that builds a dataset from r/kolkata and r/KolkataCity, covering 1 April 2025 to the present, on top of raw data fetched primarily from Arctic Shift, a free, authentication-free, continuously updated Reddit archive, after an initial batch was collected via BrightData's commercial Datasets API before that source's account quota was exhausted.

Raw JSONL responses, cached per subreddit and month, are converted into five processed tables — posts, comments, cross-posts, per-user participation, and per-month activity — through a set of scripts that re-derive every table from scratch on each run, so that collection gaps can be closed or new subreddits added without any manual table surgery. A validation step cross-checks these tables against the collection-state ledger to report duplicate IDs, missing collection periods, and collection failures, rather than assuming correctness from a successful run. As of this report, the processed dataset comprises 104,827 posts and 743,126 comments across both subreddits, 98,855 unique participating users, and zero duplicate post or comment IDs detected across repeated, independently resumed collection runs.

This report documents the task specification this pipeline was built against, the schema and deduplication design of the processing layer, the resulting dataset's validation statistics, and the practical challenges of processing several-hundred-megabyte CSV outputs and reconciling user activity across two subreddits with very different volumes. This report documents the task specification the pipeline was built against, the processing and validation methodology, the resulting dataset statistics, and the engineering challenges specific to correctness-checking a dataset at this scale.

The remainder of the report is organized as follows. Section 1 introduces why a processing and validation layer matters as much as collection itself. Section 2 states the aims and scope of this mid-term phase. Section 3 reviews the literature that informed the schema and validation design. Section 4 details the processing pipeline, deduplication strategy, and validation methodology. Section 5 presents the resulting dataset's statistics. Section 6 documents the practical challenges encountered. Sections 7 and 8 close with future work and concluding remarks, followed by a supporting appendix and the reference list.

1. Introduction to Dataset Processing and Validation

1.1. Raw API Data Is Not a Dataset

A raw JSONL dump of API responses is not, by itself, something a researcher can run an analysis against. Reddit's API-shaped JSON carries inconsistent fields across posts, deleted and removed content masquerading as ordinary rows, and no guarantee that a given post or comment was not fetched twice across two overlapping collection windows. Turning this raw material into a dataset that can be trusted requires a processing layer that imposes a fixed schema, removes duplicates by a stable identifier, and a validation layer that actively checks the result rather than simply trusting that the collection scripts ran correctly.

1.2. Why Validation Is a First-Class Concern

For a multi-month, resumable collection project, correctness bugs are easy to introduce silently: an interrupted run, a retried request, or a later run with an extended date range can each introduce duplicate records or leave an undetected gap in coverage if the processing and validation layer does not actively check for them. Treating validation as an afterthought risks discovering such problems only when a downstream analysis produces an inexplicable result, long after the data was collected.

1.3. This Project

This project is concerned with the processing and validation layer built on top of the raw data collected from r/kolkata and r/KolkataCity. It converts cached raw API responses into five de-duplicated, schema-consistent CSV tables, and produces a validation report that checks those tables against the project's own collection-state ledger for duplicate IDs, missing periods, and collection failures, so that the dataset's completeness and correctness claims are backed by an explicit, re-runnable check rather than an assumption.

2. Aims and Scope

This mid-term phase is scoped around the processing, derived-table, and validation requirements of the project's task specification, built on top of raw post and comment data for r/kolkata and r/KolkataCity collected from 1 April 2025 to the present. The objectives of this mid-term phase are:

1. **Define a Stable Processed Schema:** Convert raw, API-shaped JSONL into fixed-schema posts, comments, and cross-post tables, retaining every metadata field specified in the task brief (deletion/removal status, stickied status, score, flair, and so on) without silently dropping fields the raw API happens to omit for some records.

2. **Deduplicate by Reddit ID:** Ensure that no post or comment appears twice in the processed tables even when its collection window was fetched more than once across separate, resumed runs, using Reddit's own `post_id`/`comment_id` as the deduplication key.

3. **Build Derived Participation Tables:** From the processed posts and comments alone, with no additional API calls, construct a per-user participation table (posts/comments per subreddit, first-seen/last-seen timestamps) and a per-month activity table, and report user overlap between the two subreddits.

4. **Validate the Result:** Cross-check the processed tables against the collection-state ledger to report, per subreddit, total posts/comments, unique authors, earliest/latest post, cross-posts, deleted/removed counts, duplicate IDs, missing collection periods, and collection failures.

5. **Keep Processing Idempotent:** Ensure every processing and validation script can be re-run from scratch at any time, against whatever raw data currently exists on disk, without manual intervention or partial-state bugs, so that closing a collection gap or adding a new subreddit requires no change to the processing code.

Out of scope for this mid-term phase: any downstream content analysis of the processed text (sentiment, topic modelling, or similar), reconciling user identities across Reddit-wide history beyond what this project's own two subreddits contain, and closing the one known collection gap in the underlying raw data (comments for r/kolkata, June 2025 through December 2025), which is a collection-layer task rather than a processing or validation one and is discussed further in Section 7.

3. Literature Review and Knowledge Acquisition

3.1. Pushshift and the Archive-First Approach to Reddit Data

The methodological starting point for this project's data source is Baumgartner et al. (2020), *"The Pushshift Reddit Dataset"*, published at ICWSM, which established the practice of archiving Reddit content independently of Reddit's own API so that researchers are not subject to its rate limits or access restrictions. Arctic Shift, the data source this project's raw data is drawn from, continues this approach after changes to Reddit's API policy in 2023 disrupted Pushshift's own ingestion.

Key Takeaway for This Report: An archive-first data source shifts the correctness burden downstream — since the archive itself may contain gaps, inconsistencies, or stale records, the processing and validation layer built on top of it, which is this report's focus, cannot assume the raw data is already clean. This motivated treating validation as an explicit, separate step rather than folding a few sanity checks into the collection scripts themselves.

3.2. Idempotent, Re-Derivable Processing

A central design choice in this project's processing layer is that every processed table is re-derived from the full set of cached raw files on every run, rather than being incrementally patched. This follows a general data-engineering principle: a pipeline stage that can always be re-run from its inputs to produce the same output is easier to reason about and debug than one that accumulates state through incremental patches, because a bug in an incremental update can leave the output permanently inconsistent with its inputs in a way that is hard to detect later.

3.3. Deduplication by Stable Identifier

Using Reddit's own `post_id`/`comment_id` as the deduplication key, rather than a derived or positional key, follows directly from these identifiers being globally stable and assigned by Reddit itself at creation time — any two fetches of the same post, however far apart in time or however overlapping their collection windows, return the same ID. This is the same identifier-stability property that made Pushshift-style archives usable as a merge target for data collected across multiple, independently-run sessions in the first place.

4. Processing and Validation Methodology

4.1. Raw Data as the Single Source of Truth

Every processed table in this project is derived exclusively from the raw JSONL files cached under `raw/<subreddit>/<month>/` during collection. Collection itself used two sources over the project's lifetime: BrightData's Datasets API, which supplied the project's earliest raw data until the account's request quota was exhausted, after which collection moved to Arctic Shift, a free, authentication-free archive with no such quota, which supplied the remainder and is the source of essentially all data in the processed tables (the BrightData path is retained in the collection code but unused in practice). No processing or validation script issues a network request; all of them operate purely on what is already on disk, which keeps a full re-processing run fast and makes the raw cache the single, authoritative source of truth for every downstream table.

4.2. Posts and Cross-Posts

`process_posts.py` reads every month's raw posts file for both subreddits and emits `posts.csv` with a fixed column schema — subreddit, post ID, timestamp, author, title, post text, flair, post type, score, upvote ratio, comment count, permalink, external URL/domain, cross-post status, stickied status, and deletion/removal status — filling in `null` rather than omitting a column when a given raw record lacks a field. Every post flagged as a cross-post additionally emits a row in `crossposts.csv`, carrying the original post's ID, subreddit, author, URL, and date where available, without triggering any additional collection of the originating subreddit.

4.3. Comments

`process_comments.py` reads the two-level comment tree already reconstructed during collection (top-level comments plus their direct replies; deeper reply chains are not collected, as documented in the project README) and flattens it into `comments.csv`, one row per comment, with `depth` values of 0 or 1 and `parent_id` preserved so that a depth-1 reply tree can be reconstructed directly from the processed table without re-parsing raw data.

4.4. Deduplication by Reddit ID

Both `process_posts.py` and `process_comments.py` deduplicate by Reddit's own `post_id`/`comment_id` while appending rows, so that a post or comment collected twice (because two collection runs' date windows overlapped, or because a crash caused a partial re-fetch) is written to the processed table exactly once, regardless of how many times it appears across the raw JSONL cache.

4.5. Derived Tables: Users and Monthly Activity

`build_users.py` scans the processed posts and comments tables once to build `users.csv`: one row per username, with per-subreddit post and comment counts and first-seen/last-seen

timestamps, plus a console summary of users active in both subreddits, only r/kolkata, and only r/KolkataCity. `build_monthly_activity.py` performs the analogous aggregation by calendar month rather than by user, producing `monthly_activity.csv` with posts, comments, unique active users, and newly-first-seen users per subreddit per month. Both scripts operate purely on the processed tables, not the raw cache, and are re-run in full whenever the underlying posts or comments table changes.

4.6. Validation Against the Collection-State Ledger

`validate.py` is the correctness check for the whole pipeline: it re-scans `collection_state.csv` (the per-`(subreddit, data_type, month)` ledger maintained during collection) alongside the processed tables, and reports, per subreddit, the headline statistics in Section 5, together with any duplicate post/comment IDs found, any month flagged as other than `complete` in the state ledger (rendered as a human-readable missing-period string), and every row in `collection_state.csv` with status `failed` or `partial`, recorded verbatim as a collection failure. This step is what turns "the collection scripts ran without crashing" into an explicit, checkable claim about what was and was not actually collected.

5. Dataset and Validation Results

5.1. Processed Table Sizes and Headline Statistics

Table 1 reports the validation statistics produced by `validate.py` for both subreddits (generated 1 October 2026). The processed comment table is approximately 226 MB and the post table approximately 74 MB; across both subreddits, the pipeline has produced 104,827 processed post rows and 743,126 processed comment rows, with zero duplicate IDs detected in either table.

Table 1: Validation statistics for both subreddits (generated 2026-10-01).

Metric	r/kolkata	r/KolkataCity
Total posts	91,328	13,499
Total comments	566,456	176,670
Unique authors	71,883	38,808
Earliest post	2025-04-01	2025-04-14
Latest post	2026-09-26	2026-09-25
Cross-posts	0	93
Posts with external URLs	24	0
Deleted/removed posts	12,577	1,073
Deleted/removed comments	38,945	19,820
Duplicate post/comment IDs	0	0

5.2. User Participation Across Subreddits

`build_users.py` produced 98,855 unique usernames across both subreddits. Of these, 11,836 users post or comment in both r/kolkata and r/KolkataCity, 60,047 are active only in r/kolkata, and 26,972 are active only in r/KolkataCity. Relative to each subreddit's own author count from Table 1 (71,883 for r/kolkata, 38,808 for r/KolkataCity), the cross-subreddit overlap represents roughly 16% of r/kolkata's author base and roughly 30% of r/KolkataCity's, consistent with r/KolkataCity functioning as a smaller, more tightly overlapping community relative to the larger, more general-purpose r/kolkata.

5.3. Discussion

The zero-duplicate-ID result in Table 1, reproduced consistently by `validate.py` across repeated runs on top of a raw cache that was itself built from many separate, independently resumed collection sessions, is the main correctness evidence this processing layer provides: it confirms that the deduplication-by-Reddit-ID strategy (Section 4.4) holds in practice, not merely in design, under exactly the kind of overlapping, interrupted collection history this dataset actually has.

6. Challenges and Strategic Optimizations

6.1. Reconciling Inconsistent Raw Schemas

Raw records collected via Arctic Shift do not all carry the same set of fields — a field such as `upvote_ratio` or `stickied` may be present on a post fetched recently but absent on one fetched earlier in the project, before that field was wired into the collection client, or on the small number of posts originally sourced via the now-unused BrightData path, before that source's account quota was reached and collection moved to Arctic Shift. Rather than dropping a column whenever any record lacked it, `process_posts.py` fills missing fields with `null` on a per-record basis, so the processed schema stays fixed across the whole table and a consumer of `posts.csv` can distinguish "this field is genuinely absent for this record" from "this column does not exist."

6.2. Deduplicating Across Independently Resumed Runs

Because collection for this project happened across many separate sessions, often resumed after an interruption or extended with a later end date, the raw JSONL cache contains a meaningful number of posts and comments fetched more than once. Verifying that deduplication actually collapses these correctly — rather than merely trusting the dedup logic — required deliberately cross-checking `validate.py`'s duplicate-ID count against spot checks of the raw cache for known-overlapping collection windows, since a silent dedup failure would not have caused any visible error, only an inflated and incorrect processed table.

6.3. Processing Very Large Tables Without Loading Them Fully Into Memory

With the comment table alone around 226 MB, early versions of the processing scripts that materialized the full table in memory before writing it out were noticeably slower and more memory-hungry than necessary. Restructuring both `process_posts.py` and `process_comments.py` to append rows to disk incrementally as each raw file is read, rather than building one large in-memory structure first, kept processing within the memory budget of the development machine and made re-processing after a collection gap is closed considerably faster.

6.4. Interpreting a Validation Report That Flags Expected Gaps

`validate.py` reports every non-`complete` row in the collection-state ledger as a missing period or collection failure, including the nine months of r/kolkata comments that were deliberately never attempted as a scope decision (Section 6.3 of the companion collection-focused report) rather than collected and lost. Distinguishing a genuinely unexpected gap from an intentionally deferred one required cross-referencing the validation report against the project's own scope decisions each time it was regenerated, rather than treating every reported gap as a bug to fix immediately.

7. Future Work

As the project advances beyond the mid-term review, the following directions are planned for the processing and validation layer specifically:

- **Automated Gap Classification:** Extend `validate.py` to distinguish deliberately deferred collection gaps from genuinely unexpected ones automatically, rather than requiring manual cross-referencing against scope decisions each time the report is regenerated.
- **Schema Versioning:** Introduce an explicit schema version field in the processed tables, so that a future change to the set of retained columns does not silently change the meaning of an already-published `posts.csv` or `comments.csv`.
- **Cross-Validation Against a Second Deduplication Pass:** Implement an independent, brute-force deduplication check over the full raw cache as a periodic audit, to catch any edge case the streaming deduplication logic in `process_posts.py`/`process_comments.py` might miss.
- **Processing-Time Benchmarking:** Measure and report the wall-clock cost of a full re-processing run as the raw cache grows, to anticipate when the current single-pass, streaming-write design will need to be replaced by a more incremental approach.
- **Extending Derived Tables to New Subreddits:** Once the pipeline is extended to additional subreddits (Section 7 of the companion collection-focused report), verify that `build_users.py` and `build_monthly_activity.py` correctly extend their output schema (new `posts_<subreddit>`/`comments_<subreddit>` columns) without requiring any code change, as designed.

8. Conclusion

This mid-term phase built and validated the processing layer that turns raw, API-shaped Reddit data for r/kolkata and r/KolkataCity into a clean, de-duplicated dataset: five processed tables derived entirely from cached raw data with no re-querying of any API, and a validation step that checks the result against the project's own collection-state ledger rather than assuming correctness. As of this report, the processed dataset comprises 104,827 posts and 743,126 comments across both subreddits, 98,855 unique participating users with a documented overlap of 11,836 users active in both communities, and zero duplicate post or comment IDs detected despite a long, repeatedly interrupted and resumed collection history.

The central engineering result of this phase is that the deduplication-by-Reddit-ID strategy and the re-derive-from-scratch processing design (Section 4) hold up under exactly the conditions they were designed for — overlapping, independently resumed collection windows — as confirmed by `validate.py`'s zero-duplicate finding rather than assumed from the pipeline's design alone. The immediate next steps are to make the validation report distinguish deliberate collection-scope decisions from genuine gaps automatically, and to extend the same processing and validation design to additional subreddits as the collection layer grows, as outlined in Section 7.

Beyond the specific numbers, this phase demonstrates that treating data validation as a first-class, independently re-runnable pipeline stage, rather than an assumption built into the collection scripts, is what makes it possible to state this dataset's completeness and correctness as a checked fact rather than a hope — a distinction that matters considerably more once the dataset is handed off to downstream analysis that will not re-verify these properties itself.

9. Appendix

A.1 Processed Table Schema Summary

Table 2 summarizes the five processed tables produced by this pipeline and the script responsible for each.

Table 2: Processed tables and their generating scripts.

Table	Script	Key fields
`posts.csv`	`process_posts.py`	subreddit, post_id, timestamp, author, title, score, flair, permalink, deletion/removal status
`crossposts.csv`	`process_posts.py`	post_id, original_post_id, original_subreddit, original_author, original_url
`comments.csv`	`process_comments.py`	subreddit, post_id, comment_id, parent_id, depth (0/1), author, score
`users.csv`	`build_users.py`	username, per-subreddit post/comment counts, first_seen, last_seen
`monthly_activity.csv`	`build_monthly_activity.py`	month, subreddit, posts, comments, unique_active_users, new_users

A.2 User Overlap Between Subreddits

Table 3 reports the user-overlap breakdown computed from `users.csv`.

Table 3: User overlap between r/kolkata and r/KolkataCity.

Category	Users
Total unique users (either subreddit)	98,855
Users active in both subreddits	11,836
Users active only in r/kolkata	60,047
Users active only in r/KolkataCity	26,972

A.3 Processing Environment

Table 4 summarizes the software environment used to produce and validate the processed dataset reported above.

Table 4: Software environment for the processing and validation layer.

Setting	**Value**
Raw data source consumed	Arctic Shift JSONL cache (`raw/<subreddit>/<month>/`)
Processed comment table size	≈226 MB
Processed post table size	≈74 MB
Deduplication key	Reddit `post_id` / `comment_id`
Comment tree depth retained	2 levels (top-level + direct reply)
Validation cross-check source	`collection_state.csv`
Core libraries	Python 3, `pandas`, `NumPy`

10. References

1. Baumgartner, J., Zannettou, S., Keegan, B., Squire, M., & Blackburn, J. (2020). *The Pushshift Reddit Dataset*. Proceedings of the International AAAI Conference on Web and Social Media (ICWSM), 14, 830–839.

2. Heitmann, A. *Arctic Shift: A Continuously Updated Archive of Reddit Content*. GitHub repository. `https://github.com/ArthurHeitmann/arctic_shift`

3. Reddit, Inc. *Reddit API Documentation*. `https://www.reddit.com/dev/api/`

4. McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference (SciPy), 56–61.

5. Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). *Array Programming with NumPy*. Nature, 585, 357–362.

6. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media. (Idempotent, re-derivable pipeline design reference for Section 3.2 and Section 4.1.)